iOS Sale Item Service Guide

The Merchant Sale Item Service enables merchants to manage their sale items by offering operations to create, retrieve, update and delete items.

Below are the key utility features available:

- **Create Merchant Sale Items**: Processes the Create Sale Item request and creates the specified sale items for the merchant.
- **Get All/Specific Sale Items**: Processes the Get Sale Item request and retrieves the specified sale items for the merchant. This can be used to fetch details of a specific sale item or the complete list of sale items associated with the requested user.
- **Update Existing Sale Items**: Processes the Update Sale Item request and updates the specified sale items for the merchant.
- **Delete Merchant Sale Items**: Processes the Delete Merchant Sale Items request and then removes the requested sale item for the merchant.

Prerequisites

Before you continue, ensure you have the following prerequisites completed and ready:

- **Merchant Client Application Credentials and Access Token:** Ensure that merchant onboarding is complete and that you've received your client application credentials and access token before proceeding.
- **Access to Deluxe Developer Portal:** This portal provides additional resources, API documentation, and integration tools.
- **Merchant Authorization & Accounts Guide:** This step is required to obtain the merchant authorization token and retrieve associated merchant accounts, which are necessary to establish a connection with the terminal.
  - iOS Merchant Authorization & Accounts Guide

ON THIS PAGE

Prerequisites

Create Merchant Sale Items

Get All/Specific Sale Items

Update Existing Sale Items

Delete Merchant Sale Items

Create Merchant Sale Items

Processes the Create Sale Item request and creates the specified sale items for the merchant.

STEP 1: Create a MerchantSaleItemService Object

```
let merchantTaxService = MerchantSaleItemService()
```

STEP 2: Create Merchant Sale Items

Construct a CreateSaleItemRequest by providing the required input parameters, including saleItem: a list of MerchantSaleItem objects containing details of the sale items to be created, such as sale item ID, product name, price, category, category ID, and other relevant information.

```
// MerchantSaleItem with all fields populated
let saleItem = MerchantSaleItem(
  saleItemID: 1234,
  productName: "Veggie Burger",
  price: 5.99,
  category: "Main Course",
  categoryID: 12,
  defaultQuantity: 1,
  isvariableAmount: false,
  sku: "VB123",
  taxID: "TAX456",
  saleItemImage: "base64EncodedImageString",
  isActive: true,
  option1: nil,
  saleItemImageURL:
"https://example.com/images/veggie_burger.png"
)
```

accessToken: This should be the access token for the merchant account selected by the user. Available merchant accounts can be retrieved using the MerchantAccountService.

```
merchantTaxService.createMerchantSaleItems(request:
request, bearerToken: bearerToken) { response,
error in
if let result = response {

} else if let error = error {

}
}
```

The **bearerToken** should be the token obtained from the MerchantAuthorizationService.

CreateMerchantSaleItemResponse object which contains the following:

- **If success = true**, response contains list of MerchantSaleItem objects that were successfully created.
- **If success = false**, the response indicates a failure, and the corresponding reasons can be found in the errorCode and message fields.

On failure, the method returns an error object that describes the cause of the failure.

Get All/Specific Sale Items

Processes the Get Sale Item request and retrieves the specified sale items for the merchant. This can be used to fetch details of a specific sale item or the complete list of sale items associated with the requested user.

STEP 1: Create a MerchantTaxService Object

```
let merchantTaxService = MerchantSaleItemService()
```

STEP 2: Get Merchant Sale Items

The GetSaleItemRequest object is used to define the data required to fetch sale item entries for a merchant account, where saleItemId is the unique identifier of the sale item to be retrieved—if provided, only that specific item's details will be fetched; otherwise, all sale items for the merchant will be returned.

```
let request = GetSaleItemRequest(
accessToken: "761f497a-65c4-489e-973d-
79ab84fd3011",
saleItemId: 501
)
```

accessToken: This should be the access token for the merchant account selected by the user. Available merchant accounts can be retrieved using the MerchantAccountService.

Pass the request and bearer token to the getMerchantSaleItems() method.

```
else if let error = error {

}
}
```

The **bearerToken** should be the token obtained from the MerchantAuthorizationService.

STEP 3: Response Handling

On success, getMerchantSaleItems() method returns a GetMerchantSaleItemsResponse object containing the following:

- **If success = true**, response contains list of MerchantSaleItem objects that were successfully fetched.
- **If success = false**, the response indicates a failure, and the corresponding reasons can be found in the errorCode and message fields.

On failure, the method returns an error object that describes the cause of the failure.

Update Existing Sale Items

Processes the Update Sale Item request and updates the specified sale items for the merchant.

STEP 1: Create a MerchantSaleItemService Object

```
let saleItemService = MerchantSaleItemService()
```

STEP 2: Update Merchant Sale Item

The UpdateSaleItemRequest object is used to define the data required to update sale items for a merchant account, including a list of MerchantSaleItem objects containing details of the sale items to be updated.

```
let updateRequest = UpdateSaleItemRequest(
accessToken: "761f497a-65c4-489e-973d-
79ab84fd3011",
saleItem: [MerchantSaleItem1, MerchantSaleItem2]
)
```

Pass the request and bearer token to the updateMerchantSaleItems() method.

```
merchantTaxService.updateMerchantSaleItems(request:
request, bearerToken: bearerToken) { response,
error in
if let result = response {

} else if let error = error {

}
}
```

The **bearerToken** should be the token obtained from the MerchantAuthorizationService.

STEP 3: Response Handling

On success, updateMerchantSaleItems() method returns a CreateMerchantSaleItemResponse object containing the following:

- **If success = true**, response contains list of MerchantSaleItem objects that were successfully updated.
- **If success = false**, the response indicates a failure, and the corresponding reasons can be found in the errorCode and message fields.

On failure, the method returns an error object that describes the cause of the failure.

Delete Merchant Sale Items

Processes the Delete Merchant Sale Items request and then removes the requested sale item for the merchant.

STEP 1: Create a MerchantSaleItemService Object

```
let saleItemService = MerchantSaleItemService()
```

STEP 2: Delete a Merchant Sale Item

The GetSaleItemRequest object is used to define the data required to delete sale item entries for a merchant account,

```
let request = GetSaleItemRequest(
accessToken: "761f497a-65c4-489e-973d-
79ab84fd3011",
saleItemId: 501
)
```

accessToken: This should be the access token for the merchant account selected by the user. Available merchant accounts can be retrieved using the MerchantAccountService.

Pass the request and bearer token to the deleteMerchantSaleItems method.

```
saleItemService.deleteMerchantSaleItems(request:
request, bearerToken: bearerToken) { response,
error in
if let result = response {
} else if let error = error {

}
}
```

The **bearerToken** should be the token obtained from the MerchantAuthorizationService.

STEP 3: Response Handling

On success, deleteMerchantSaleItems() method returns a DeleteSaleItemResponse object which contains the following:

- **If success = true**, response contains list of MerchantSaleItem objects that were successfully deleted.
- **If success = false**, the response indicates a failure, and the corresponding reasons can be found in the errorCode and message fields.

On failure, the method returns an error object that describes the cause of the failure.

Note: *Please refer to the Complete SDK Reference for additional details iOS SDK Reference Guide*

deluxe. connect

Payments API

Deluxe.com | Privacy Policy | Do Not Sell My Personal Information

Deluxe Corporation, 801 S Marquette Ave,
Minneapolis, MN 55402.

© 2026 Deluxe Corporation